

LIFETIMES

Definition

In Rust, **every reference** has a lifetime, which represents the period during which the value that the reference points to **exists in memory**.

A variable's lifetime begins when it is declared, and ends when it goes out of scope.

Rust's lifetimes are a way to ensure **memory safety** **without overhead** at the **small cost** of being explicit about lifetimes in the face of ambiguity.

Reminder on references

A **reference** in Rust (&) is a way to **borrow** a value without taking ownership of it.

Exercise 0: References reminder

Variables scopes and lifetimes

Definition

Variable scopes define:

- **Where** a variable can be used
- **When** a variable is created and destroyed

In Rust, a variable is valid **from the point it is declared** until it **goes out of scope**. When a variable goes out of scope, Rust automatically cleans up its memory, calling `drop()`.

Key Difference: Scope vs Lifetime

Scope: Where a variable or reference is valid.

Lifetime: How long a reference remains valid relative to the data it borrows.

Example: Basic Variable Scope



```
fn main() {  
    {  
        let x = 5; // x is valid from this point.  
        println!("x is: {}", x);  
    }  
    // x is now out of scope and dropped.  
    // println!("x is: {}", x); // ERROR: x does not exist here.  
}
```

Exercise 2: Lifetimes for variables

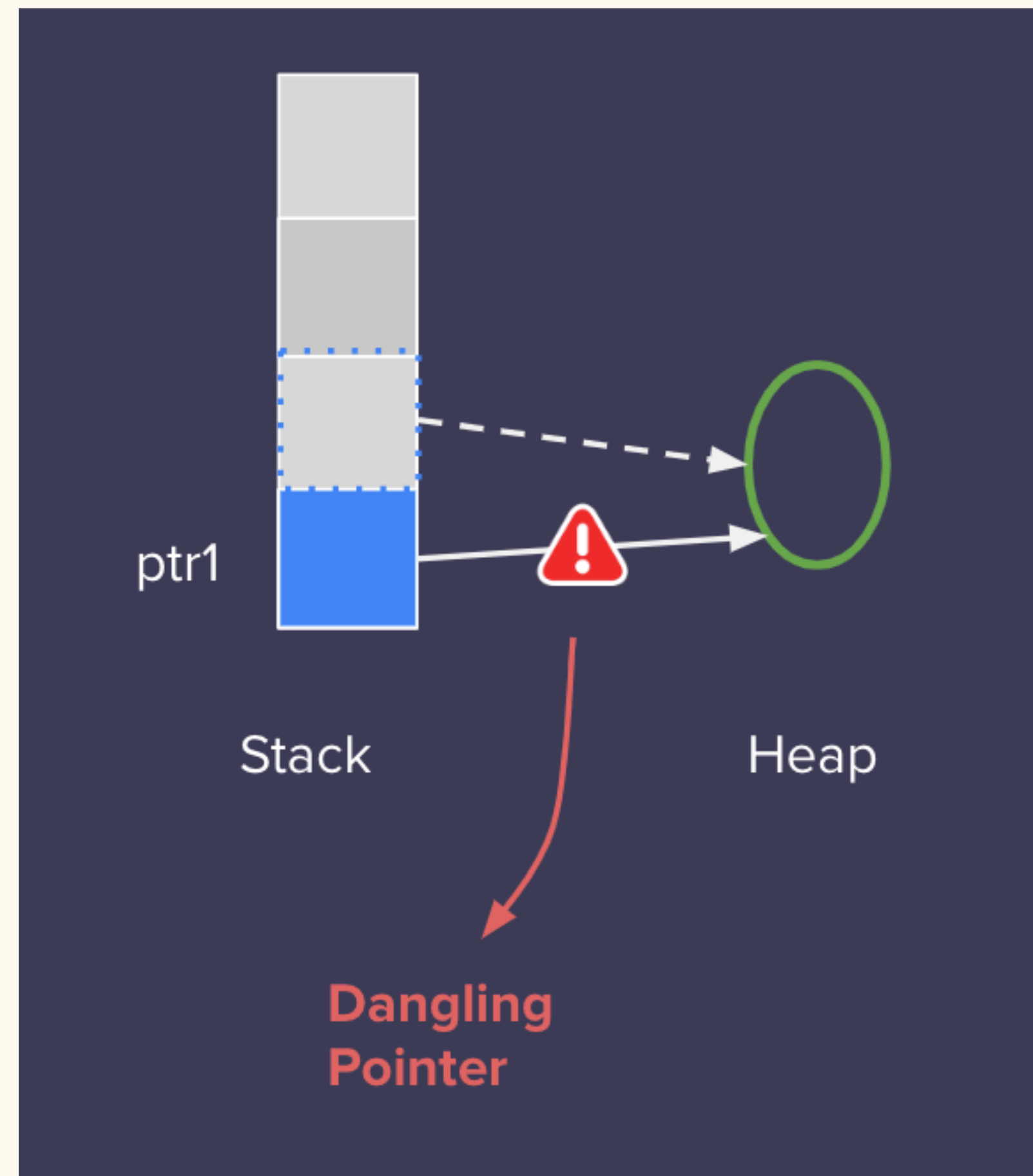
Static lifetimes

The **'static lifetime** is a special case where data is valid for the **entire program duration**. It is used for global constants or string literals.



```
let s: &'static str = "I have a static lifetime.";
```

Dangling references



A dangling reference is a reference to data that **no longer exists**.

When you use references (&) in Rust, you **borrow** data instead of taking ownership. However, **references must obey scope rules too**: the reference cannot outlive the data it points to.

Example: Ownership and Scope

```
fn main() {  
    let s1 = String::from("hello");  
    {  
        let s2 = &s1; // s2 borrows s1  
        println!("{}", s2);  
    } // s2 goes out of scope, but s1 is still valid.  
    println!("{}", s1); // s1 can still be used.  
}
```


Example: Ownership and Scope ❌

```
fn main() {  
    let r;  
    {  
        let x = 5;  
        r = &x; // ERROR: x goes out of scope here.  
    }  
    println!("{}", r); // r is invalid because x is no longer valid.  
}
```

Exercise 3: Dangling references

Generic lifetimes (with functions)

Generic lifetimes in Rust are **placeholders** that tell the compiler **how long references live** in relation to each other.

Example: **'a** is a generic lifetime used to say « this reference must live **at least as long as 'a** ».

Example with 'a



```
fn get_first<'a>(text: &'a str) -> &'a str {  
    &text[0..1]  
}
```

Example in the Rust book

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() {  
        x  
    } else {  
        y  
    }  
}  
  
fn example_with_helper() {  
    let string1 = "abcd";  
    let string2 = "xyz";  
  
    let result = longest(&string1, &string2);  
    println!("The longest string is {}", result);  
}
```

Return values checked against annotations in signature

Lookup lifetime annotations in type signature to determine how long reference should live for

Exercise 4:

Generic lifetimes 'a in function

Your turn!

Generic Lifetimes vs. Concrete lifetimes

- A **concrete lifetime** is the region of code that a value exists in the program (the time it lives in its memory address).
- A **generic lifetime** is more abstract. It is a hypothetical lifetime, a non-specific lifetime, a future lifetime that can vary.
- We can annotate generic lifetimes in our code. This enables functions that are flexible enough to handle varying lifetimes.

Plein

Lifetime Annotations

- A **lifetime annotation** is a name or label for a lifetime.
- Lifetime annotations don't change the reference's lifetime. They don't affect the logic in any way.
- A lifetime annotation is a piece of metadata that we provide to the borrow checker so that it can validate that references are valid.

Sous-t


```
{  
  let coffees: [String; 2] = [String::from("Latte"), String::from("Mocha")];  
  let  
  pri  
}
```

First Elision Rule: The compiler assigns a lifetime to each parameter that is a reference.

Sous-ti

```
let cities: Vec<String> = vec![  
  String::from("London"),  
  String::from("New York"),  
  String::from("Barcelona"),  
];  
let two  
let  
sel  
};  
println  
{  
  let  
  let  
  pri  
}
```

Second Elision Rule: If there is one reference parameter and the return value is a reference, the borrow checker will infer that their lifetimes are related.

Third Elision Rule: In a method definition, if there are multiple reference parameters but one of them is **self**, the borrow checker will assume the lifetime of the instance is connected to the lifetime of the return value.

Sous-ti

1:14 / 14:57

The **borrow checker** is the part of the Rust compiler that validates that all borrows (i.e., references) are valid.

Common Lifetime Errors

1. Returning References to Local Variables

Rust won't allow you to return a reference to a value that goes out of scope:

```
fn invalid_reference() -> &str {  
    let s = String::from("hello");  
    &s // ERROR: `s` is dropped here  
}
```

Solution: Return an owned type (e.g., String) instead:

```
fn valid_reference() -> String {  
    let s = String::from("hello");  
    s  
}
```

nightlybin+lib

rustowl

Visualize Ownership and Lifetimes in Rust

by cordx56 and 18 contributors

Install

API reference

GitHub repo (cordx56)

Documentation (github.com)

9 releases

Uses new Rust 2024

0.3.4

May 20, 2025

0.3.3

May 17, 2025

0.3.0

Apr 30, 2025

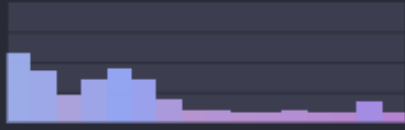
0.2.2

Apr 18, 2025

0.1.4

Feb 22, 2025

#250 in Development tools



416 downloads per month

MPL-2.0 license

3MB

4K SLoC



 RustOwl

Visualize ownership and lifetimes in Rust for debugging and optimization

main.rs 1, U

Cargo.toml U

設定

src > main.rs > main > mut str_b

Run | Debug

1fn main() {

2let str_a: String = String::from("a");

3let mut refer: &String = &str_a;

4if refer == "a" {

5let mut str_b: String = String::from("b");

6str_b.push_str(string: refer);

7refer = &str_b;

8

9

10println!("{refer}");

11}

12

variable str_b is required to live here

main*

1 0

Live Share

Git Graph

rust-analyzer

✓ RustOwl

-- NORM

Lifetime Subtyping and Polymorphism

Rust supports lifetime subtyping and polymorphism. Lifetime subtyping means that one lifetime can be a subset of another.

```
fn longest<'a>(x: &'a str, y: &str) -> &'a str {  
    x  
}
```

In this example, the first input parameter has a lifetime `'a`, while the second input parameter does not have an explicit lifetime annotation. This means the second input parameter can have any lifetime, and it will not affect the return value.



```
//error: `y` does not live long enough
{
    let x: &Vec<i32>;
    {
        let y = Vec::new();//-----+
//                                     | y's lifetime
//                                     |
        x = &y;//-----+-----+
//                                     |
    }// <-----+                                     | x's lifetime
    println!("x's length is {}", x.len());// |
}// <-----+
```



```
let red = "Short";  
let green = "Very long";  
  
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() {  
        x  
    } else {  
        y  
    }  
}
```

```
fn longer_string<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() {  
        x  
    } else {  
        y  
    }  
}
```



```
{
    let r;                                // -----+-- 'a
                                         //      |
    {                                     //      |
        let x = 5;                       // -+-- 'b  |
        r = &x;                           //  |    |
    }                                     // -+    |
                                         //      |
    println!("r: {}", r);                 //      |
}                                         // -----+
```

YOUR TEXT GOES HERE